

Bảng phân công công việc — Đồ án BrewLite

Ứng dụng đặt cà phê không dùng tiền mặt · Quy trình Agile Scrum
Môn: Công nghệ Phần mềm · Nhóm 2 thành viên · Lập ngày: 05/10/2026 (bản sửa đổi 2)
Công nghệ: Next.js (Frontend) · NestJS (Backend) · PostgreSQL + Prisma · Docker Compose

1. Thành viên và vai trò

Thành viên	Vai trò Scrum	Mảng phụ trách	Trách nhiệm chính
Long	Scrum Master + Development Team	Backend (NestJS)	Giữ nhịp Sprint, chủ trì các buổi họp, cập nhật Backlog và Burndown. Thiết kế và công bố hợp đồng API, xây dựng API, cơ sở dữ liệu và nghiệp vụ đơn hàng
Tài	Development Team	Frontend (Next.js)	Xây dựng toàn bộ giao diện người dùng: menu, chi tiết sản phẩm, giỏ hàng, màn thanh toán và xác nhận đơn
Giảng viên	Product Owner	—	Xác định yêu cầu, xếp ưu tiên Product Backlog, nghiệm thu Increment sau mỗi Sprint

2. Phân công 10 task theo Product Backlog

#	Tên task	Tiêu chí chấp nhận chính	Người làm	Sprint	Điểm
1	Khởi tạo dự án và cấu trúc	Hai thư mục frontend + backend, chạy được "Hello", có README, Git, .env.example	Cả hai	1	1
2	API danh sách sản phẩm	GET /products trả JSON menu: id, tên, giá, ảnh	Long	1	1
3	Trang Menu	Gọi /products, render lưới sản phẩm, có trạng thái loading và empty	Tài	1	1
4	Chi tiết và tùy chọn sản phẩm	Màn chi tiết: chọn size S/M/L và topping, tính giá theo tùy chọn	Tài	2	1
5	Giỏ hàng	Thêm / sửa số lượng / xóa, lưu state bằng Zustand, tự tính tổng tiền, badge số lượng	Tài	2	1
6	API tạo đơn hàng	POST /orders nhận giỏ hàng, validate bằng class-validator, lưu trạng thái PENDING, trả mã đơn	Long	2	1
7	Đăng ký / Đăng nhập (JWT)	POST /auth/register, /auth/login, băm mật khẩu bcrypt, phát JWT, guard bảo vệ route đặt đơn	Long	2	1
8	Thanh toán không tiền mặt	Mock payment: nhận Ví / Thẻ, POST /payments, đơn chuyển PAID khi thành công, PAYMENT_FAILED khi lỗi	Long	3	1
9	Xác nhận, lịch sử đơn và bàn giao	Màn xác nhận, GET /orders/me, docker-compose chạy cả 3 service, README, demo end-to-end	Cả hai	3	1
10	Nghiệp vụ backend	Ưu tiên Order State Machine và test chặn chuyển trạng thái sai. Idempotency, optimistic locking, loyalty nếu còn thời gian	Long	3	1
				Tổng	10

Hợp đồng API — việc phải làm xong đầu tiên

Long công bố hình dạng JSON của /products và /orders ngay trong 20 phút đầu Sprint 1, ghi vào docs/API-CONTRACT.md. Tài dựng giao diện với dữ liệu giả theo đúng hình dạng đó, không chờ backend hoàn thành. Mọi thay đổi hợp đồng phải báo người kia trước khi sửa.

3. Kế hoạch 3 Sprint

Sprint	Thời gian	Mục tiêu (Sprint Goal)	Task
Sprint 1	Tối ngày 1 21:00 – 00:00	Mở trình duyệt thấy được menu quán, dữ liệu lấy từ API thật	Task 1, 2, 3
Sprint 2	Sáng ngày 2 07:30 – 12:00	Chọn được món với size và topping, bỏ vào giỏ, đặt đơn thành công	Task 4, 5, 6, 7
Sprint 3	Chiều ngày 2 13:00 – 17:30	Thanh toán hoàn tất, chạy được luồng end-to-end, sẵn sàng bàn giao	Task 8, 9, 10

Lịch làm việc chi tiết

Khung giờ	Long — Backend	Tài — Frontend
Sprint 1 — Menu xem được		
21:00 – 21:20	Sprint Planning. Dựng bảng Product Backlog. Long công bố hợp đồng API.	
21:20 – 22:00	Task 1 — khởi tạo repo, hai thư mục, chạy "Hello", README, commit.	
22:00 – 23:30	Task 2 — GET /products	Task 3 — trang Menu, dùng dữ liệu giả
23:30 – 00:00	Ghép frontend với API thật. Sprint Review. Commit.	
Sprint 2 — Chọn món, giỏ hàng, đặt đơn		
07:30 – 07:45	Daily Scrum — hôm qua xong gì, hôm nay làm gì, đang vướng gì.	
07:45 – 09:30	Task 6 — POST /orders , validate, PENDING	Task 4 — chi tiết, size, topping, tính giá
09:30 – 11:15	Task 7 — JWT, bcrypt, guard	Task 5 — giỏ hàng, tổng tiền, badge
11:15 – 12:00	Ghép, Sprint Review, commit.	
Sprint 3 — Thanh toán và bàn giao		
13:00 – 14:30	Task 8 — mock payment, PAID / PAYMENT_FAILED	Màn thanh toán và màn xác nhận đơn
14:30 – 16:00	Task 10 — State Machine + test chặn chuyển sai	Màn lịch sử đơn — gọi GET /orders/me
16:00 – 17:30	Task 9 — docker-compose chạy cả 3 service, hoàn chỉnh README.	
Bàn giao		
18:00 – 19:00	Chạy thử end-to-end 3 lần từ máy sạch. Sửa lỗi phát sinh.	
19:00 – 19:45	Rà Definition of Done cho từng task. Task nào thiếu thì bù hoặc đánh dấu chưa Done.	
19:45 – 20:30	Viết Sprint Retrospective. Dọn repo. Commit cuối.	

4. Definition of Done

Một task chỉ được tính điểm khi thỏa đủ sáu điều kiện sau. Thiếu một điều kiện là chưa Done.

- Code chạy được, không lỗi build.
- Đáp ứng đúng tiêu chí chấp nhận của task.
- Có commit trên Git với mô tả rõ ràng, được người còn lại review.
- API có validate đầu vào.
- Giao diện xử lý trạng thái loading và lỗi cơ bản.
- README có hướng dẫn chạy.

5. Quy ước làm việc

Hạng mục	Nhóm thống nhất
Quy ước nhánh	Mỗi task làm trên nhánh riêng: <tên-người>/task-N-mo-ta-ngan . Ví dụ: long/task-2-api-san-pham , tai/task-3-trang-menu . Không push thẳng vào main.
Quy ước commit	Tiếng Việt có dấu, định dạng <loại>(task-N) : <mô tả> . Loại: feat, fix, chore, docs, refactor, test, style. Ví dụ: feat(task-2): thêm endpoint lấy danh sách sản phẩm
Tần suất commit	Commit ngay sau mỗi task hoàn thành. Không gom nhiều task vào một commit.
Review chéo	Xong task thì mở Pull Request, người kia đọc và approve rồi mới merge vào main.
Độ dài Sprint	Cố định, không kéo dài giữa chừng. Làm không kịp thì cắt bớt task, đẩy về Backlog.
Thay đổi phạm vi	Không nhận thêm việc mới giữa Sprint. Ghi vào Backlog, bàn ở buổi Planning kế tiếp.

Rủi ro đã nhận diện

Thời gian rất gấp (khoảng 24 giờ). Nguyên tắc ưu tiên: **làm ít task nhưng mỗi task đạt Definition of Done**, vì task làm dở không được tính điểm. Nếu không đủ thời gian, ba yêu cầu còn lại của Task 10 (idempotency, optimistic locking, loyalty) được bỏ trước tiên, sau đó tới Task 7 (JWT). Phạm vi đã làm được trình bày trung thực khi bàn giao.

Long — Scrum Master, Backend	Tài — Development Team, Frontend
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên)